Athens University of Economics and

Business Department of Informatics

COURSE: **PARALLEL PROGRAMMING**

Spring Semester 2025-26

ASSIGNMENT TITLE

Experimenting Different Methods for Workload Distribution with CUDA/OpenCL in C++

Name: MARAKIS MICHAIL
Semester: 6th
Email: p3230267@aueb.gr
Prof: Dr. VASILAKIS ANDREAS-ALEXANDROS

Table of Contents

CUDA

Classifications

- The function I will be using for this assignment is $f(x) = x^2$.
- The GPU that these experiments are being operated on is **NVIDIA GeForce GTX 1650.** The PC crashes when the algorithm is running for N > 10^8. It is worth mentioning that for N = 10^2 the integral result was 333.35 which is very close to the actual result of 333 333. Also, for 10^3 the result is 333.334 which is even closer to the actual result, but the execution times are very low for both cases (e.g. 6.31e-05 seconds) and hard to make observations out of. As a result, the experiments are taking place with **N = 10^8** and **N = 10^5** where execution times are easier to deal with and can make clear observations.
- **GPU calculates everything and nothing is being operated on CPU. The code on every experiment gives the best and most productive results based on my hardware.**

Workload Distribution experiment

N = 10^8
Result = 333.333

Distribution	Avg Time of Execution
1 element per thread	0.0457874
Multiple elements per thread	0.0249364

According to the results, the distribution that allows each thread to process

multiple elements is more efficient than assigning one element per thread, achieving a speedup of approximately **1.83x**. This improvement occurs because each thread performs more useful computation, reducing thread management overhead and improving overall GPU utilization. As a result, computational resources are used more effectively, leading to better overall performance.

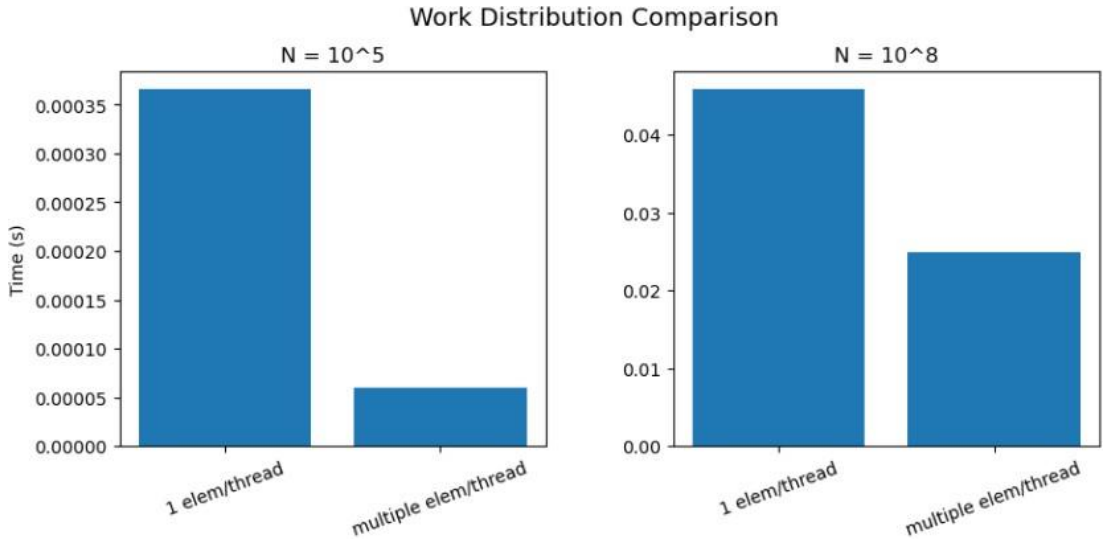
N = 10^5
Result = 333.333

Distribution	Avg Time of Execution
1 element per thread	0.0003655
Multiple elements per thread	5.95e-05

For the same reason as when N = 10^8, the Multiple elements per thread case is way faster than the 1 element per thread case.

NOTES:

- Tests took place using different local work sizes (e.g. 32, 64, 128, 256) and there was no difference in which method was faster. As we can see, the N made that difference. Times were faster when local work size = 128 and local work size = 256. The tables above show the average times when local work size = 256. Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between times for different N)

According to the graphs above, it is observable that multiple elements per thread strategy consistently outperforms the one element per thread approach. This behavior is observed for both problem sizes. However, the speedup varies dramatically with N, increasing from approximately **1.83x for N=10^8 to 71.6x for N=10^5.**

This difference indicates that workload distribution affects performance differently depending on the problem size. For smaller N, thread management overhead becomes more significant, making the multi-element case more efficient, as it reduces the number of active threads and improves resource utilization. For larger N, both approaches achieve better GPU utilization, reducing the relative performance

gap.

Different Threads per Block Experiment

N = 10^8
Result = 333.333

Block Size	Avg Time of Execution
32	0.0461142
64	0.0246175
128	0.0237451
256	0.0248886
512	0.0229351

From the results for large N, it is observed that the execution time varies slightly across different block sizes. The best performance is achieved when block size = 512, with the lowest average execution time (0.0229). When block size = 32, it shows a significantly higher execution time (0.0461) because such a small block size fails to fully utilize the GPU's hardware resources, leaving computing units idle. On the other hand, for block sizes from 64 up to 512, the performance remains very stable and close to the optimal limit, proving that the GPU reaches its hardware limit point early on and maintains high efficiency across larger block cases.

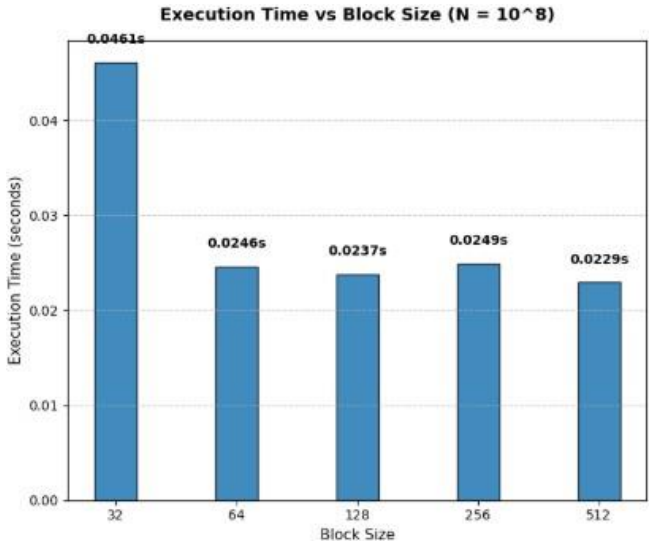
N = 10^5
Result = 333.333

Block Size	Avg Time of Execution
32	0.0003883
64	5.99e-05
128	5.94e-05
256	5.8e-05
512	5.01e-05

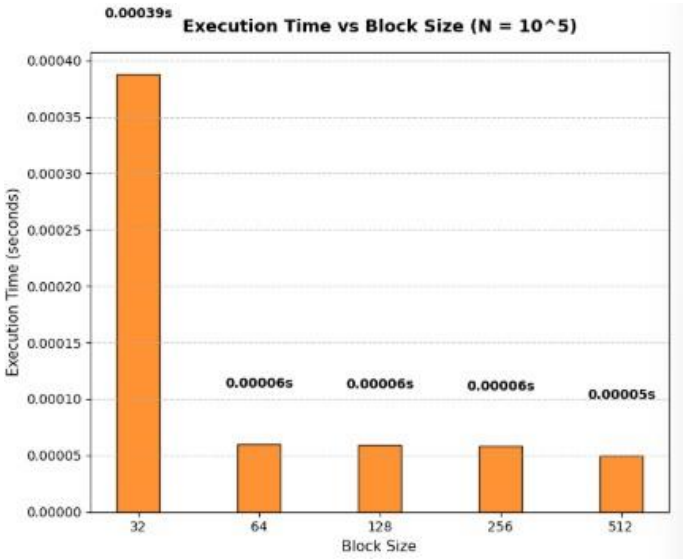
For small N, the execution times are generally lower, as expected due to the smaller N. The best performance is again observed when block size = 512, where the lowest average execution time is achieved (5.01e-05). Apart from the worst case of block size = 32, which again shows significantly higher execution time (0.0003883) due to under-utilization of the GPU resources, the differences between the other block sizes are extremely small. This indicates that once the block size is 64 or larger, the GPU instantly reaches its hardware limit point, making the performance highly stable across all remaining cases for small workloads.

NOTES:

- Results are presented using mult elem/thread and with a block barrier. Changing the hyperparameters didn’t give as much impact as the block size and N gave on this algorithm. Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between block sizes for N = 10^8)



(Comparison between block sizes for N = 10^5)

From the comparison of both graphs, it is observable that block size has a stronger impact on performance for smaller values of N. For N = $\mathbf{10^5}$, the difference between the worst case and the rest is way bigger while for N = $\mathbf{10^8}$ the difference is much smaller but still very noticeable.

Barrier Experiment

N = 10^8
Result = 333.333

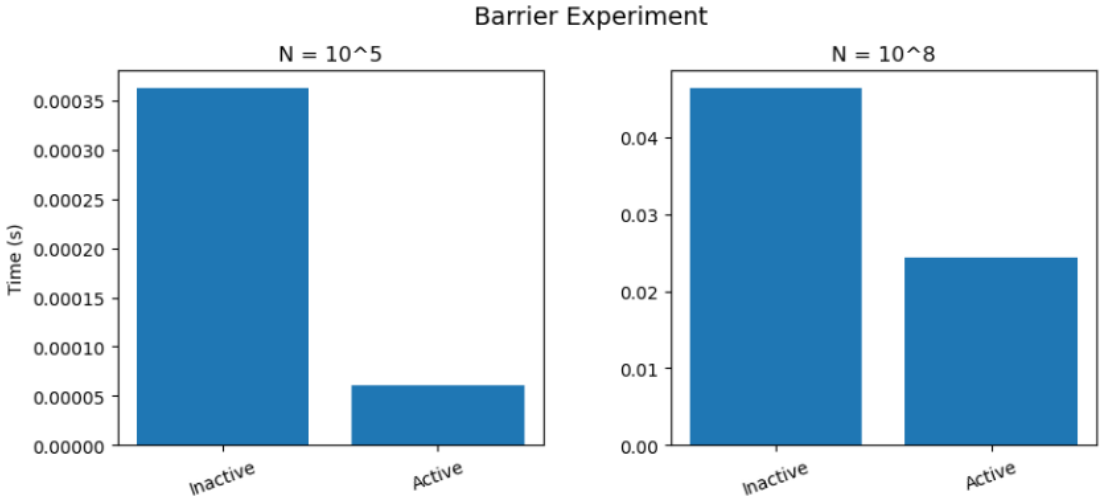
Barrier status	Avg Time of Execution
Inactive	0.046362
Active	0.0243787

According to the results, the case with the barrier active shows slightly faster execution time for N = 10^8, with a speedup of approximately **1.9x** compared to the inactive barrier case. The difference is relatively small but still noticeable. This happens because the barrier makes all threads in a block wait until they finish their work before continuing. In this way, the work inside each block is better synchronized before writing results to global memory, in this case at the partial_sum dynamic array

N = 10^5
Result = 333.333

Barrier status	Avg Time of Execution
Inactive	0.0003624
Active	6.11e-05

For the same reason as the case of N = 10^8, having an active barrier makes the algorithm more efficient. The difference here is the speedup where the active barrier case is approximately **7x faster**.



(Comparison between barrier status for different N)

From the graphs we can clearly observe the difference in execution time between the two cases. For the smaller value of N, the impact of the barrier is more noticeable, since synchronization overhead becomes more significant compared to the actual computation. For the larger N, this effect is reduced because the computation dominates the execution time, making the difference between the two cases less visible.

NOTES:

- I tested both strategies of global barrier using it and without using it, on different local work sizes and I got the same results. The cases of 128 and 256 have by far the best results from the smaller local work sizes and the 512 cases. On the other hand, I tested the algorithms on different workload distributions as well and again it doesn't have the impact that N has on the algorithm. For smaller N, 1 elem/thread is by far faster where for bigger N the mult elem/thread is much more productive. The results are presented with local work size = 256

and with 1 elem/thread. Analytical results of the impact of each hyperparameter are analyzed on the respected experiment

Different Memory Models

N = 10^8
Result = 333.333

Memory Model	Avg Time of Execution
Registers	0.0909231
Global	0.0498088
Shared	0.0484792

For large N, **Shared Memory** is the fastest because the heavy tree reduction happens inside the GPU's fast block memory and only one thread per block writes to the main memory, minimizing data traffic. **Global Memory** is almost identical in execution time because threads read consecutive numbers, allowing the GPU to automatically group these reads together and load data in fast batches. On the other hand, **Registers** are by far the slowest(**nearly 2 times slower than the rest**) because they are strictly private to each thread since the reduction loop requires threads to constantly share and exchange numbers, the GPU is forced to copy data out to the slow local memory so the threads can communicate, which creates severe delays.

N = 10^5
Result = 333.333

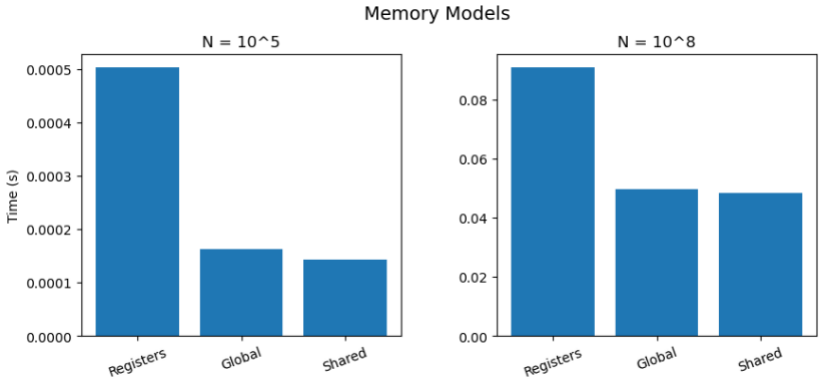
Memory Model	Avg Time of Execution
Registers	0.000503
Global	0.0001633
Shared	0.0001437

For small N, execution times are much lower due to the smaller workload, with **Shared Memory** remaining the fastest and **Global Memory** following closely, while **Registers** are again by far the slowest. In this case Registers are nearly 3.1 times slower than the other strategies. Shared Memory is the most efficient because the tree reduction happens entirely within the GPU's fast block memory, requiring only one global write per block. Global Memory performs nearly as fast because the GPU automatically merges the consecutive memory reads into quick batches which keeps the small workload efficient. On the other hand, Registers are the slowest because they are strictly private to each thread and since the reduction loop forces threads to constantly share numbers, the GPU must waste time copying data out to the slow local memory so threads can communicate, which creates a huge overhead that slows down this small workload significantly.

NOTES:

- I Tested every memory model algorithm with different block size, different workload distribution and different barrier status and the main observation was that changing these secondary parameters had no significant impact on the relative performance. Shared Memory consistently remained the fastest, followed by Global Memory, while Registers always performed the worst. Results are presented with block size = 256 with workload distribution of 1 elem/thread and without the use of global barrier with __syncthreads().

- Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between memory models for different N)

According to the graphs, it is also obvious that execution times for Global and Shared do not differ much from each other whereas the productivity for Registers is very low as execution times are way higher than the rest.

OpenCL-GPU

Classifications

- The GPU that these experiments are being operated on is **Intel(R) UHD Graphics 620.** The PC crashes when the algorithm is running for N > 10^8. It is worth mentioning that for N = 10^2 the integral result was 333.35 which is very close to the actual result of 333.333. Also, for 10^3 the result is 333.334 which is even closer to the actual result, but the execution times are very low for both cases (e.g. 5.34e-08 seconds) and hard to make observations out of. As a result, the experiments are taking place with **N = 10^8** and **N = 10^5** where execution times are easier to deal with and can make clear observations.
- My laptop's GPU has an upper constraint of maximum work-group size of 256. As a result, any attempt to launch a kernel with a larger local work size violates the OpenCL execution constraints defined by my hardware and is rejected by the runtime. Experiments with local work size = 512 are therefore not executable and are reported as **GPU LIMITATION ERROR** in the experiments of different local work sizes.

```
Device ID: 0
Name: Intel(R) UHD Graphics 620
Compute Units: 24
Global Memory: 6.34 GB
Local Memory (shared): 64 KB
Max Work-Group Size: 256
Max Work Item Sizes: (256, 256, 256)
Max Dimensions: 3
Clock Frequency: 1100 MHz
Vendor: Intel(R) Corporation
```

- **GPU calculates everything and nothing is being operated on CPU. The**

code on every experiment gives the best and most productive results based on my hardware.

Workload Distribution experiment

N = 10^8
Result = 333.333

Distribution	Avg Time of Execution
1 element per thread	0.21275
Multiple elements per thread	0.0496492

According to the results, the distribution that allows each thread to process multiple elements is much more efficient than assigning one element per thread, achieving a **speedup** of approximately **4.285x**. This improvement occurs because each thread performs more useful computation, reducing thread management overhead and improving overall GPU utilization. As a result, computational resources are used more effectively, leading to better overall performance.

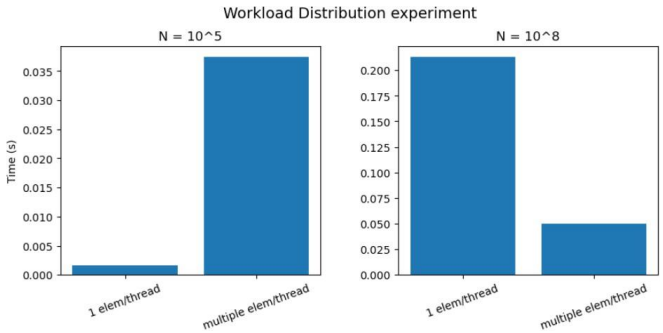
N = 10^5
Result = 333.333

Distribution	Avg Time of Execution
1 element per thread	0.0016729
Multiple elements per thread	0.037384

On the other hand, we can see that with lower N, using the 1 element per thread distribution is **22.35** times faster than the other case. This is logical as with lower N the impact initializing the loop boundaries at every single iteration step has more impact as work is much less.

NOTES:

- Tests took place using different local work sizes (e.g. 32, 64, 128, 256) and there was no difference in which method was faster. As we can see, the N made that difference. Times were faster when local work size = 128 and local work size = 256. The tables above show the average times when local work size = 256.
- Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between times for different N)

From the graphs above we can visually observe the importance of N in this

case. To be more specific as N is lower the more impact the initializing of the loop has and thus the algorithm is slower with multiple elements per thread.

Different Threads per Block Experiment

N = 10^8
Result = 333.333

Local Work Size	Avg Time of Execution
32	0.0993762
64	0.0428977
128	0.0412651
256	0.0418046
512	GPU LIMITATION ERROR

From the results for N = 10^8, it is observed that the execution time does not vary much across the different local work sizes. The best performance is achieved when local work size = 128, with the lowest average execution time. When local work size = 32 it is shown to have much higher execution time due to increased scheduling overhead, while the cases of 64 and 256 have almost identical execution times with when local work size = 128. When local work size = 512 the GPU does not execute the kernel due to its resource's limitations, having block size of 512 is way too heavy for it. It is interesting that the fastest case is **2.4x** faster than the slowest case

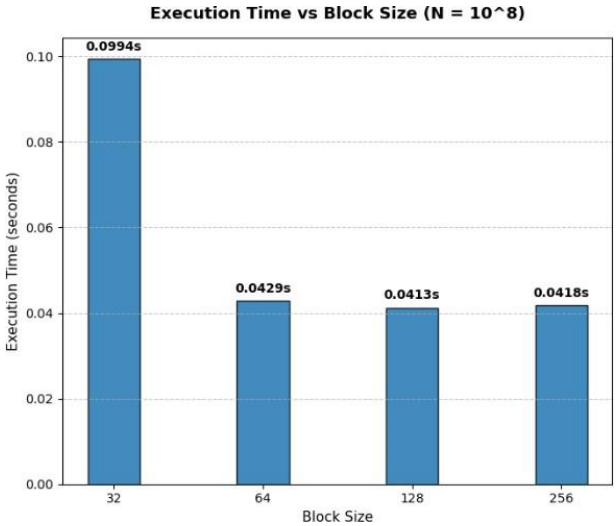
N = 10^5
Result = 333.333

Local Work Size	Avg Time of Execution
32	0.0020181
64	0.0005735
128	0.0004429
256	0.0004944
512	GPU LIMITATION ERROR

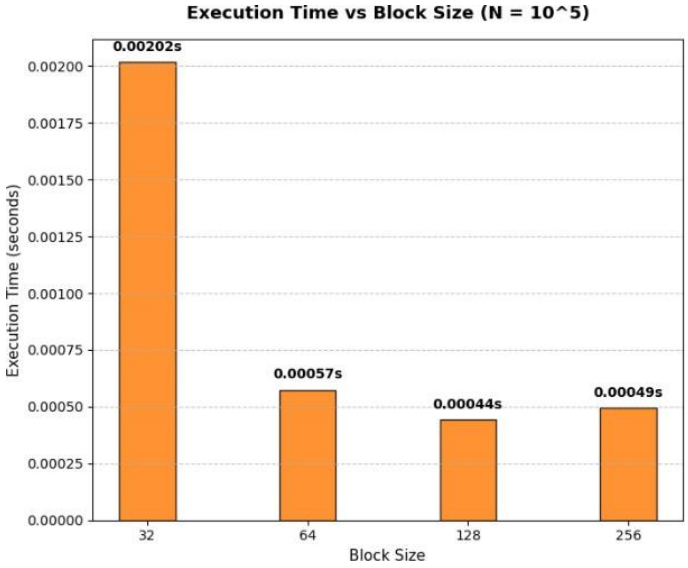
For N = 10^5, the execution times are generally lower, as expected due to the smaller N. The best performance is again observed when block size = 128, where the GPU achieves a good balance between resources and scheduling overhead. Apart from the worst case of local work size = 32, differences between the other local work sizes are a bit bigger compared to the larger N case but a difference that is noticeable. It is interesting that the fastest case is **4.55x** faster than the slowest case

NOTES:

- As I mentioned in the different workload distribution experiments, local work size = 256 and local work size = 128 give the best results in both cases of the workload distribution methods without having any major difference. Results are presented using 1 elem/thread.
- Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between local work sizes for N = 10^8)



(Comparison between local work sizes for N = 10^5)

According to the visual presentation we can see that N doesn't have big impact on the differences between execution times per local work size. The cases of 64, 128 and 256 have relatively same execution times, with 128 being slightly faster than the rest. On the other hand, the case of local work size = 32 is by far the worst and we can observe the difference as well between the speedups from both cases.

Barrier Experiment

N = 10^8
Result = 333.333

Barrier status	Avg Time of Execution
Inactive	0.130262
Active	0.0486596

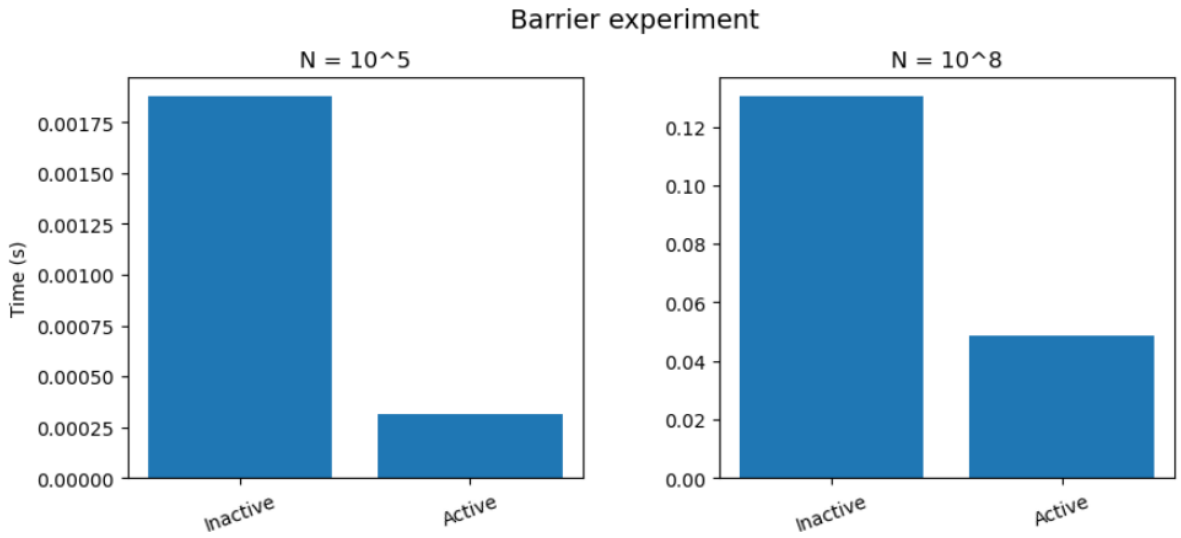
According to the results, the case with the barrier active shows slightly faster execution time for large N, with a speedup of approximately **2.267x** compared to the

inactive barrier case. This happens because the barrier makes all threads in a block wait until they finish their work before continuing. In this way, the work inside each block is better synchronized and organized in perfectly aligned groups before writing results to global memory at the partial_sum dynamic array. Without the barrier, the threads execute out-of-sync, creating a lot of traffic and confusion in the memory access which slows down the inactive case.

N = 10^5
Result = 333.333

Barrier status	Avg Time of Execution
Inactive	0.0018763
Active	0.0003114

For the same reason as the case of N = 10^8, having an active barrier makes the algorithm more efficient. The difference here is the speedup where the active barrier case is **6.02x faster**.



(Comparison between barrier status for different N)

From the graphs we can clearly observe the difference in execution time between the two cases. For the smaller value of N, the impact of the barrier is more noticeable, resulting in a higher speedup of **6.02x** compared to **2.267x** for the larger N. This happens because under lower workloads, the structured thread organization enforced by the active barrier helps the GPU run much more efficiently without losing time in loose scheduling delays. For the larger N, this effect is reduced because the massive size of the computation saturates the GPU anyway, making the execution behavior more stable and the difference between the two cases less visible

NOTES:

- I tested both strategies of global barrier using it and without using it, on different local work sizes and I got the same results. The cases of 128 and 256 have by far the best results from the smaller local work sizes. On the other hand, I tested the algorithms on different workload distributions as well and again it doesn't have the impact that N has on the algorithm. For smaller N, 1 elem/thread is by far faster where for bigger N the mult elem/thread is much more productive.

The results are presented with local work size = 128 and with 1 elem/thread.

- Analytical results of the impact of each hyperparameter are analyzed on the respected experiment

Different Memory Models

N = 10^8
Result = 333.333

Memory Model	Avg Time of Execution
Registers	0.0052498
Global	0.0022735
Shared	0.0021948

For large N, **Shared Memory** is the fastest because the heavy tree reduction happens inside the GPU's fast block memory and only one thread per block writes to the main memory, minimizing data traffic. **Global Memory** is almost identical in execution time because threads read consecutive numbers, allowing the GPU to automatically group these reads together and load data in fast batches. On the other hand, **Registers** are by far the slowest(**nearly 2 times slower than the rest**) because they are strictly private to each thread since the reduction loop requires threads to constantly share and exchange numbers, the GPU is forced to copy data out to the slow local memory so the threads can communicate, which creates severe delays.

N = 10^5
Result = 333.333

Memory Model	Avg Time of Execution
Registers	0.0027583
Global	0.00074
Shared	0.0006353

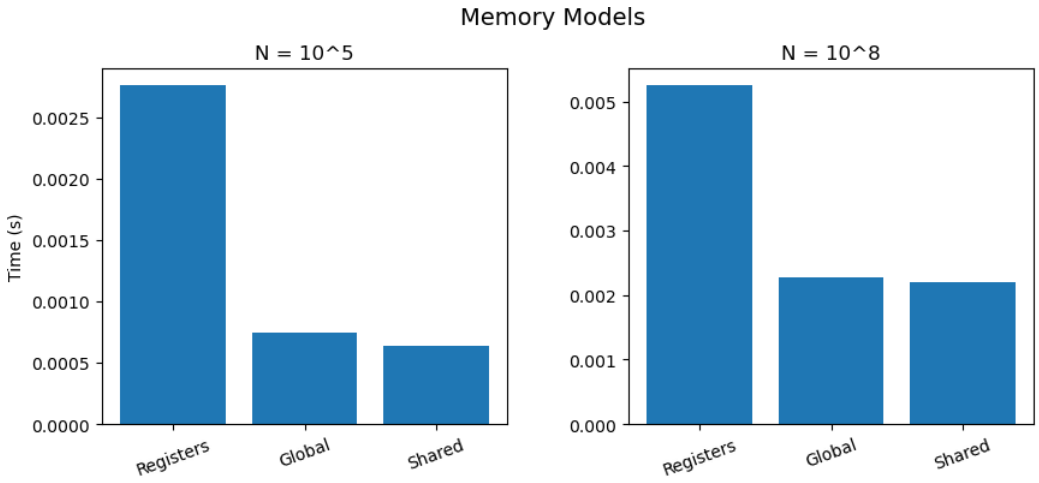
For small N, execution times are much lower due to the smaller workload, with **Shared Memory** remaining the fastest and **Global Memory** following closely, while **Registers** are again by far the slowest. In this case Registers are nearly 3.1 times slower than the other strategies. Shared Memory is the most efficient because the tree reduction happens entirely within the GPU's fast block memory, requiring only one global write per block. Global Memory performs nearly as fast because the GPU automatically merges the consecutive memory reads into quick batches which keeps the small workload efficient. On the other hand, Registers are the slowest because they are strictly private to each thread and since the reduction loop forces threads to constantly share numbers, the GPU must waste time copying data out to the slow local memory so threads can communicate, which creates a huge overhead that slows down this small workload significantly.

NOTES:

- I Tested every memory model algorithm with different block size, different workload distribution and different barrier status and the main observation was that changing these secondary parameters had no significant impact on the relative performance. Shared Memory consistently remained the fastest, followed by Global Memory, while Registers always performed the worst.

Results are presented with local work size = 256 with workload distribution of 1 elem/thread and with the use of global barrier.

- Analytical results of the impact of each hyperparameter are analyzed on the respected experiment



(Comparison between memory models for different N)

According to the graphs, it is also obvious that execution times for Global and Shared do not differ much from each other whereas the productivity for Registers is very low as execution times are way higher than the rest.

pthreads Vs OpenMP Vs CUDA Vs OpenCL

According to all my assignments the fastest methods on **N = 1000000 for f(x) =** x^2 were

- For **OpenMP** was Guided Routing with **avg execution time = 0.119524**
- For **pthreads** was Cyclic Scheduling with **avg execution time = 0.0760965**.
  For **CUDA** was when using Multiple elements per thread with block size = 512, using a global barrier and with the global memory model with **avg execution time = 0.0229351**
- For **OpenCL** was when using Multiple elements per thread, with shared memory model with block size = 256 with global barrier with **avg execution time = 0.0021948**

It is observable that OpenCL with these hyperparameters is the most productive way of calculating this integration. Second best is the CUDA implementation. This is expected as these algorithms operate on GPU whereas the other implementations are working on CPU.

For reference, it is interesting to note that the **sequential algorithm** has an **avg execution time of 0.565095.** We can observe the efficiency that parallelism offers in a computational problem like this one. Statistically, OpenMP, which is the slowest of the parallel methods, is about **5x faster** than the sequential algorithm.